ISO 26430-6 审计与链路安全

用一个可运行原型讲清标准的骨架与边界

这不是一份抽象导读，而是把条款、线协议、TLS 约束、命令语义和当前实现一一对上。

例子来源

仓库实现了 ISO 26430-

这份标准在管什么

Positioning

标准对象

- 面向 D-Cinema 场景中的 **Auditorium Security Messages**
- 规定 Initiator 和 Responder 之间的安全消息传输方式
- 重点不是“业务流程”，而是 **跨设备互操作的消息规范**

标准结构

- 第 6 章：TLS、KLV、通用元素、RRP 约束
- 第 7 章：通用命令
- 第 8 章：Link Encryption 命令
- Annex A：每类命令对应的 UL

读法建议：把这份标准理解为“在 TLS 通道中传输一组固定 KLV 包格式的命令集合”。项目代码正好能把这三层拆开看清楚。

从标准目录看主线

Outline

第 6 章

- 6.1 TLS 约束
- 6.2 KLV 固定长包
- 6.3 Request_ID、Response、Batch
- 6.4 同步 RRP、端口 1173、互认证

第 7 至 8 章

- 7.1 BadRequest Response
- 7.2–7.5 GetTime / Event / QuerySPB
- 8.1–8.5 LEKeyLoad / Query / Purge

TLS → KLV → RRP → 命令语义

1. 传输与安全层

第 6 章先规定消息跑在什么样的通道里

6.1 TLS 约束

TLS

标准要求

- TLS 1.0
- 双向证书认证
- 唯一 cipher suite: `TLS_RSA_WITH_AES_128_CBC_SHA`
- 仅支持 RSA 2048 / exponent 65537
- 无压缩
- TLS record size = 512 bytes

本项目对应

- `src/security/tls_profile.cpp`
- `src/security/certificate_utils.cpp`
- `tests/test_tls.cpp`

已实现：TLS 1.0、单一 cipher、禁压缩、互认证、最小 RSA 约束。
未实现：完整 D-Cinema 证书 profile、512-byte record、显式 ChangeCipherSpec 特殊处理。

代码例子：TlsProfile 如何把条款落地

Code

来自 src/security/tls_profile.cpp

```
expect(SSL_CTX_set_cipher_list(ctx, "AES128-SHA"), ...);
expect(SSL_CTX_set_min_proto_version(ctx, TLS1_VERSION), ...);
expect(SSL_CTX_set_max_proto_version(ctx, TLS1_VERSION), ...);
SSL_CTX_set_options(ctx, SSL_OP_NO_COMPRESSION);
verify_mode = SSL_VERIFY_PEER | SSL_VERIFY_FAIL_IF_NO_PEER_CERT;
```

应该怎么理解

- 这段代码不是“OpenSSL 调参”，而是在把标准里的通信约束硬编码成互操作基线。
- 这里能直接回答“项目为何能说自己部分符合 6.1”。
- 但它也说明：只校验到 RSA/2048/65537，不等于完整满足 D-Cinema 证书规范。

6.4 RRP 约束和端口要求

RRP

标准要求

- 使用同步 **Request-Response Pair**
- 同一对 SPB 之间，一个 RRP 完成后再开下一个
- 使用 well-known port `1173`
- 忙时允许返回 busy

本项目对应

- `src/apps/asm_responder.cpp` 默认端口 1173
- `src/transport/tls_server.cpp` 每次读取一条 request 再写回一条 response
- 当前没有真正的 busy 处理路径

关键理解：ISO 26430-6 不是定义一套异步消息总线，而是定义“在 TLS socket 上做同步问答”的设备间安全控制协议。

2. KLV 与通用元素

6.2 与 6.3 决定每条消息长什么样

6.2 KLV：为什么这份标准适合做字节级测试

KLV

标准要求

Request / Response 都是 KLV 包，Key 是 16-byte UL，Length 是固定 4-byte long-form BER，Value 里只放字段值。

本项目拆分方式

- `src/klv/klv.cpp` 负责纯 KLV 编解码
- `src/asm/codec.cpp` 负责命令 UL 和字段拼接
- `tests/test_codec.cpp` 做字节级回归

带来的好处

- 标准是否被破坏，一眼看十六进制就知道
- 互操作联调时不依赖“解释正确”
- 适合做黄金向量冻结

标准帧例子：GetTime Request

Vector

来自 `docs/protocol-vectors.md`

```
060e2b34020501010207010210000000830000040000000 1
```

拆开看

- 前 16 字节：UL，对应 GetTime Request
- 接着 4 字节：`83 00 00 04`
- Value：只有 4-byte `Request_ID = 1`

标准意义

这正是 6.2 和 6.3 的结合：
Key 决定命令类型，Length 决定包长，Value 放固定字段，所有多字节整数都用 big-endian。

6.3 通用元素：Request_ID、Response、Batch

Elements

三件最重要的事

- `Request_ID` 非零，Response 必须回显
- Response code 统一为 successful / failed / invalid / busy
- Batch 一律是 8-byte 头 + 固定 item 长度 + n 个 item

代码映射

- `include/asm/common.hpp`
- `src/asm/codec.cpp`
- `src/asm/event_log.cpp`
- `src/asm/le_key.cpp`

例子

`EventIDBatch` 和 `LEKeyIDBatch` 都用同一批处理思路：先写数量，再写单项长度，再写元素序列。这就是标准把“可扩展列表”做成可预测字节布局的方法。

3. 第 7 章通用命令

这些命令和设备类型无关，是 ASM 的基本能力

7.1–7.5 通用命令一览

General

命令	标准作用	本项目实现
BadRequest	请求无法按原命令回应时的兜底响应	支持未知 UL、解码失败、`Request_ID=0`
GetTime	取 Responder 当前 UTC 秒数	返回 `Uint64 epoch`
GetEventList	按时间窗返回事件 ID 集合	返回 `EventIDBatch`
GetEventID	按 ID 取单条日志记录	当前返回 XML 文本
QuerySPB	查询 SPB 协议版本和状态	返回 `Protocol_Ver + Status`

读标准时的抓手

第 7 章真正重要的不是命令名字，而是每条命令都严格服从第 6 章定义的公共外壳。

服务层例子：AsmService 如何分发通用命令

Dispatch

来自 src/asm/service.cpp

```
switch (request.command) {
  case Command::get_time: return handle_get_time(request);
  case Command::get_event_list: return handle_get_event_list(request);
  case Command::get_event_id: return handle_get_event_id(request);
  case Command::query_spb: return handle_query_spb(request);
  ...
}
```

标准视角下的意义

- 第 6 章把“统一壳”定死，第 7/8 章只定义 payload 语义。
- 所以服务层天然长成“先识别命令，再交给各自 payload handler”的样子。
- 这也是为什么本项目能把 Codec、EventLog、LEKey 分层拆开。

GetEventList / GetEventID：标准把日志抓取拆成两步

Events

标准设计

- 先按时间窗拿 ID 列表
- 再逐条按 ID 取日志正文
- 好处是避免一次返回巨量日志文本

本项目实现

- `src/asm/event_log.cpp` 提供 `list_event_ids()` 和 `get_event_xml()`
- 默认事件样例包括：
- `Responder started`
- `TLS profile uses development CA`
- `Link encryption material not loaded`

实现边界

当前 XML 只是“对齐 Text Log Record 字段语义”的原型，不是完整 SMPTE 430-5 signed log pipeline。

4. 第 8 章 LE 命令

这部分把链路加密密钥管理做成一组明确 RRP

8.1–8.5 LE 命令一览

LEKey

命令	标准作用	本项目做法
LEKeyLoad	下发一批 LE key	32-byte item，支持 overflow 返回 failed
LEKeyQueryID	查某个 key 是否存在	返回 `KeyPresent`
LEKeyQueryAll	枚举全部活动 key ID	返回 `LEKeyIDBatch`
LEKeyPurgeID	删除一个 key	返回 `NoKeyID`，但 Response 仍 successful
LEKeyPurgeAll	删除全部 key	返回最小成功响应

关键点

标准并不定义真正的加密硬件实现，而是定义“管理接口的线协议”。这也是项目能用 mock key store 先把协议跑通的原因。

为什么 LEKeyLoad 的 batch 设计很重要

Batch

单项结构

- LE Key ID: 4 bytes
- Key: 16 bytes
- Expire Time: 4 bytes
- Attribute Data: 8 bytes

合计正好 32 bytes。

本项目对应

- `src/asm/le_key.cpp` 按标准顺序拼包
- 支持至少 16 个 key 的容量约束
- 超容量时 `Overflow != 0` 且 `Response = failed`

这页想说明什么

第 8 章不是“随便定义几个密钥接口”，而是在用严格固定长度的 item 设计，把批量装载、存在性查询和清除语义变成可互操作的线协议。

5. 用代码把标准读透

这个仓库最有价值的地方，是它把标准拆成了可测试的模块

项目模块和标准条款的对应关系

Mapping

模块	主要文件	对应标准
transport	`tls_server.cpp`, `tls_client.cpp`	6.1, 6.4
security	`tls_profile.cpp`, `certificate_utils.cpp`	6.1
klv	`klv.cpp`	6.2
asm::Codec	`codec.cpp`	6.3 + Annex A
asm::EventLog	`event_log.cpp`	7.3, 7.4
asm::QuerySPB	`query_spb.cpp`	7.5
asm::LEKey	`le_key.cpp`	8.1–8.5
asm::AsmService	`service.cpp`	第 7、8 章总分发

为什么测试在这类标准实现里特别重要

Tests

本项目测试层

- `tests/test_codec.cpp`
- `tests/test_service.cpp`
- `tests/test_tls.cpp`
- `docs/protocol-vectors.md`

它们分别证明什么

- 编码没有漂移
- 命令语义没有写歪
- TLS 约束不是文档口头声明
- 互操作联调有固定基线

这类标准实现最怕什么？最怕“看起来懂了标准，但字节布局悄悄变了”。黄金向量和集成测试正是防这个。

标准理解和当前实现边界

Boundary

已经对齐的骨架

- TLS 1.0 + 单一 cipher
- KLV 固定 4-byte BER
- Annex A UL 映射
- 第 7 章通用命令
- 第 8 章 LE 命令

仍然是原型的部分

- 完整 D-Cinema X.509 profile
- TLS 512-byte record 强制
- ChangeCipherSpec 显式特殊处理
- SMPTE 430-5 签名日志
- 真实安全模块和真实链路加密后端

所以这份项目最适合拿来做什么

适合拿来学习标准、验证字节级协议、做互操作起步样机；不应直接包装成“完整影院生产实现”。

回看这份标准，最重要的不是记住命令名

而是理解它如何把 TLS、KLV、RRP 和命令语义拼成一套可互操作的安全消息协议。

本项目的价值就在于：它把这个拼接过程写成了可运行、可测试、可检查的代码。